Making VCL fifty times faster

Harry Langford
Department of Computer Science
University of Oxford

Abstract

Variational Continual Learning is a continual learning framework based on recursive Bayesian estimation. The original paper, however, focused solely on proving the mathematical soundness of the framework, and demonstrating a proof-of-concept implementation. The literature has since taken the general proof-of-concept implementation of variational continual learning to be an upper bound of what this framework is capable of. I probe this assumption by pushing Variational Continual Learning far beyond what has previously been demonstrated, and deriving how it can be applied to regression. Through better coreset selection, and a generalisable mathematically justified choice of prior, I improve upon all previous results with VCL on classification by 2% accuracy, and am able to match previous classification performance using 2% as much compute. All code was written by me. Source code is available at `https://anonymous.4open.science/r/ConfidencePrior-AF25/`.

1 Introduction

The soundness of almost all machine learning relies on the assumption that data is independent, and identically distributed. This assumption typically does not hold on real world data, where covariate shift commonly causes an observable shift in dataset distribution. The standard solution is to collect all data, randomise the order, and consider the data distribution to be the weighted mean of all observed data distributions. This solution is inapplicable if fully retraining models after each task is computationally infeasible, if we must make predictions whilst data is being streamed, or if storing all the data at once is not feasible. Continual Learning is the problem of learning on a changing data distribution. The primary issue facing Continual Learning is catastrophic forgetting, where data distributions seen early in training is forgotten. Nguyen et al. propose Variational Continual Learning (VCL) as a solution to continual learning, and prove its mathematical soundness.

VCL has, however, not seen widespread adoption, largely due to empirical deficiencies such as computationally expensive training, difficulty selecting priors, non-uniform performance across tasks, and no demonstrated performance in other problem settings. This miniproject pushes the VCL framework as far as it goes, providing a mathematical foundation for prior selection, and an investigation of coreset selection. The combination of these achieves to a 2% accuracy improvement with 5% as much compute, achieving equal performance across all tasks, and can achieve the results in the original paper with 2% as much compute as Nguyen et al. reported. I additionally generalise VCL to regression, and identify an important limitation in this domain.

2 Background

Continual Learning is essentially an investigation of how to beat catastrophic forgetting and catastrophic remembering. These are dual problems, where models trained on multiple tasks experience catastrophic forgetting if they forget earlier tasks, and experience catastrophic remembering if remembering earlier tasks prevents them learning new tasks. Parisi et al. identified three high-level approaches to combatting these problems, although others have identified more [Wang et al., 2024].

Regularisation approaches Adding a regularisation term to the loss to restrict how parameters can change on a new task is a popular method of limiting catastrophic forgetting. This regularisation term is typically KL-divergence, or weighted ℓ_2 loss. VCL uses KL divergence [Nguyen et al., 2018], whilst EWC (in fig. 6) uses ℓ_2 loss weighted by variance [Kirkpatrick et al., 2016], and SI uses ℓ_2 loss weighted by path integrals of the gradient vector [Zenke et al., 2017].

Dynamic architectures Regularisation methods are prone to both catastrophic forgetting, and catastrophic remembering. An approach to prevent this is to explicitly allocate new weights to new tasks, making the network wider for each task, and freezing old weights when training new weights [Rusu et al., 2016]. If data distributions for tasks are sufficiently similar, new weights can exploit existing weights and lead to higher performance.

Complementary learning systems and memory replay Neither of the above methods completely solve continual learning, and all viable methods suffer from catastrophic forgetting to some extent. It is often viable to store *some* information about old tasks, meaning that information about old tasks can be replayed to the model. This replay can be in the form of replaying old training examples [Chaudhry et al., 2019], distilling information from previous versions [Rebuffi et al., 2017, Castro et al., 2018] or even generating synthetic examples for old tasks [Shin et al., 2017].

3 Methodology

VCL uses a combination of regularisation, and memory replay to achieve competitive performance. VCL takes a Bayesian view of the problem of continual learning. We define a variational distribution, and for each task, finding the variational posterior which maximises the prior multiplied by the likelihood of generating the data which was observed in task t_i. Variational continual learning repeatedly applies the following approximation:

$$\tilde{q}_{t+1}(\theta) = \arg\min_{q \in \mathcal{Q}} \mathrm{KL}\left(q(\theta) \parallel \frac{1}{Z_t}\tilde{q}_t(\theta) \cdot p(\mathcal{D}_t \mid \theta)\right)$$

This is equivalent to finding the variational posterior that maximises the sum of the crossentropy and the KL-divergence with the previous posterior. This results in the algorithm given in algorithm 1.

The variational posterior used in the original paper was a Gaussian mean-field posterior, which is trained through Bayes by Backprop [Blundell et al., 2015]. The original paper produced coresets by randomly subsampling the dataest, and the coreset would grow by n datapoints after each task.

VCL, however, has a high associated training cost, and the mathematical derivation of VCL is currently only complete for classification. This miniproject investigates how to push VCL to its limits, and demonstrates that the framework is capable of much more than it has previously achieved.

4 Extension

Variational continual learning suffers from several empirical deficiencies. The most significant are the compute cost, difficulty choosing prior, and that catastrophic forgetting is only slowed. By inspection of algorithm 1, I notice that there are three significant implementation choices: the variational posterior, the prior, and the coreset selection method. Significant research has been done into the differences between variational distributions, both model architecture and weight distribution [Sun et al., 2017]. Little research has been done into optimal prior selection, and the research into optimal coreset selection has not been applied to VCL.

My project aims to alleviate the last two of these problems. I take a principled mathematical approach to choice of prior, which I find speeds up VCL convergence twenty-fold, and improves performance after convergence; I then consider a more realistic problem setting which allows me to use a new class of coreset algorithms, which further increases rate of convergence and improves performance once again. I find that I beat the performance which the original paper achieved in a hundred epochs using only two epochs. Final convergence is achieved after five epochs, and that in a setting similar to coreset of size 200, I achieve over 2% higher accuracy.

Using a prior on accuracy to derive a prior on weights Machine learning models are black boxes: typically, our only real prior is on the performance we expect them to achieve. We have no true priors on weights, and when asked to choose priors on them, we often use arbitrary priors which form good models, rather than encode information apriori. If our model is calibrated, the mean probability it outputs is an estimator for its accuracy. Assuming our logits are normally distributed, we can use our prior on accuracy to determine the prior on our logits, $\mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$. For a model of depth d, this prior over logits implies a prior that weights are distributed according to a Kaiming-normal distribution [He et al., 2015] scaled by $\sigma^{\frac{2}{d}}$. To add a prior that biases are distributed according to $\mathcal{N}(\mathbf{0}, \sigma_b^2\mathbf{I})$ for $\sigma_b^2 < \sigma^{\frac{2}{d}}$, we instead scale the weight prior by $\sigma^{\frac{2}{d}} - \sigma_b^2$. I provide a formal derivation in section C, and demonstrate, in section 5, that using the scaled Kaiming-normal distribution as a prior leads to faster convergence, superior performance, and better stability.

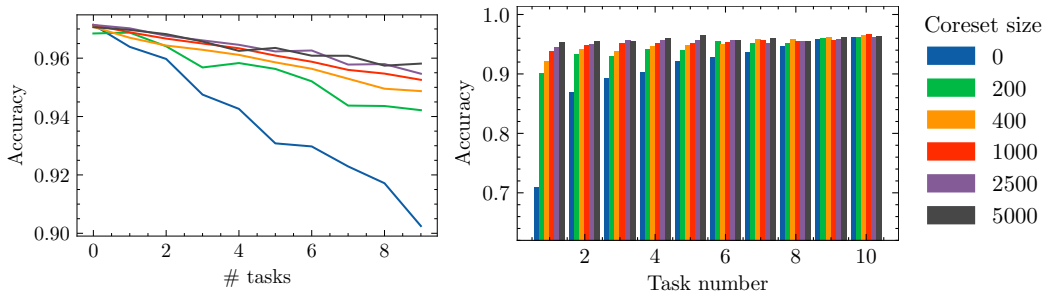
Expanding the class of coreset algorithms Nguyen et al. consider two closely related coreset selection algorithms and observe little difference. Wang et al. claims, in the context of memory replay, that the difference between selection algorithms is small, and they perform 'mediocrely'. I consider a separate problem setting which aims to exploit all memory available to the user. If the user has coresets of size n for each task and trains on t tasks, they must have storage for $n \cdot t$ examples. Given a coreset of size c, I propose that before task $t+1$, the coreset should contain $\frac{c}{t}$ examples from each task $\leq t$, then randomly evict $\frac{c}{t+1}$ examples from the coreset into the training data for task $t+1$ and add $\frac{c}{t+1}$ random examples from task $t+1$. This falls within the VCL framework. I find that this coreset algorithm leads to better performance.

Minimum required extension An additional constraint preventing VCL from being adopted is that it is only shown to work on classification tasks. My miniproject also generalises it to regression tasks. This requires a slight adaptation to the loss function, wherein we must weight the KL-divergence by the variance we expect the data to have. I provide a full derivation in section B.1. The final loss function, which we want to *minimise* is given below:

$$\mathcal{L}^t_{\mathrm{VCL}}(q_t(\theta)) = \sum_{n=1}^{N_t} \mathbb{E}_{\theta\sim q_t(\theta)} \left[\left(f\left(\mathbf{x}_t^{(n)} \mid \theta\right) - y_t^{(n)} \right)^2 \right] + 2\sigma^2 \cdot \mathrm{KL}(q_t(\theta) \parallel q_{t-1}(\theta))$$

5 Experiments

Discriminative VCL with a mathematically derived prior I investigate whether my mathematically derived prior leads to an increase in performance for VCL. I find that it does, leading to convergence to better solutions, and significantly faster convergence. The results are shown in fig. 1, which was trained for 5 epochs, as opposed to the 100 by Nguyen et al.. My prior for model confidence was 90%, whilst the weight prior used by Nguyen et al. corresponded to 99.8%.

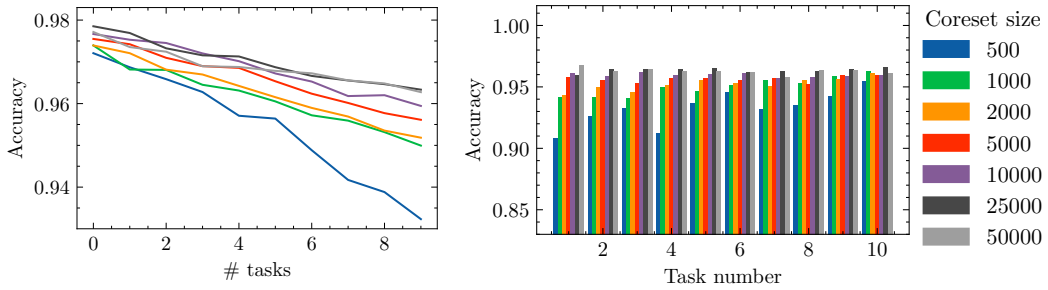


(a) Accuracy of VCL on permuted MNIST

(b) Accuracy on permuted MNIST tasks

Figure 1: Coreset VCL accuracy using a scaled Kaiming-normal prior. Increasing coreset size has diminishing returns, coresets mostly affect performance on earlier tasks.

Unified coresets exploit all space I find that my coreset method leads to higher accuracy, and more uniform performance across tasks, as seen in fig. 2. I used naïve random sampling. More advanced methods [Aljundi et al., 2019, Borsos et al., 2020] will likely yield better results.



(a) Accuracy of VCL with unified coresets on permuted MNIST

(b) Accuracy on of VCL with unified coresets on permuted MNIST tasks

Figure 2: VCL accuracy when using unified coresets and a scaled Kaiming-normal prior. Unified coresets perform better than increasing-size coresets, with more uniform performance across tasks. Note that on t tasks, a unified coreset of $n \cdot t$ corresponds to size n per-task coreset.

Minimum required extension The generalisation of VCL to Gaussian likelihood worked. The suitable standard deviation is a property of the dataset: making a suitable choice was difficult, and was done empirically. My experiments are written up in section B.2.

6 Conclusions

A mathematically motivated prior is important A mathematically motivated prior selection method is of great importance to Bayesian machine learning. The results were *better* than those found in the paper, indicating that the reported results likely hold. Understanding how generalisable this prior selection algorithm is requires a wider empirical investigation.

Opening This paper takes a novel view on how coresets can be selected, aiming to maximally utilise available memory. I find potential for further exploration, and welcome research into the tradeoff between training on data now, and training on it later.

References

R. Aljundi, M. Lin, B. Goujaud, and Y. Bengio. Gradient based sample selection for online continual learning. In *NeurIPS*, 2019.

C. Blundell, J. Cornebise, K. Kavukcuoglu, and D. Wierstra. Weight Uncertainty in Neural Networks. In *ICML*, 2015.

Z. Borsos, M. Mutny, and A. Krause. Coresets via Bilevel Optimization for Continual Learning and Streaming. In *NeurIPS*, 2020.

F. M. Castro, M. J. Marin-Jimenez, N. Guil, C. Schmid, and K. Alahari. End-to-End Incremental Learning. In *ECCV*, 2018.

A. Chaudhry, M. Rohrbach, M. Elhoseiny, T. Ajanthan, P. K. Dokania, P. H. S. Torr, and M. Ranzato. Continual Learning with Tiny Episodic Memories. *CoRR*, 2019. URL `http://arxiv.org/abs/1902.10486`.

C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger. On Calibration of Modern Neural Networks. In *ICML*, 2017.

K. He, X. Zhang, S. Ren, and J. Sun. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. In *ICCV*, 2015.

J. Kirkpatrick, R. Pascanu, N. C. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell. Overcoming catastrophic forgetting in neural networks. *CoRR*, 2016. URL `https://www.pnas.org/doi/abs/10.1073/pnas.1611835114`.

C. V. Nguyen, Y. Li, T. D. Bui, and R. E. Turner. Variational Continual Learning. In *ICLR*, 2018. URL `https://openreview.net/forum?id=BkQqq0gRb`.

G. I. Parisi, R. Kemker, J. L. Part, C. Kanan, and S. Wermter. Continual Lifelong Learning with Neural Networks: A Review. *Neural Networks*, 2019. URL `https://www.sciencedirect.com/science/article/pii/S0893608019300231`.

S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert. iCaRL: Incremental Classifier and Representation Learning. In *CVPR*, 2017.

A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell. Progressive Neural Networks. *CoRR*, 2016. URL `http://arxiv.org/abs/1606.04671`.

H. Shin, J. K. Lee, J. Kim, and J. Kim. Continual Learning with Deep Generative Replay. In *NeurIPS*, 2017.

S. Sun, C. Chen, and L. Carin. Learning Structured Weight Uncertainty in Bayesian Neural Networks. In *AISTATS*, 2017.

L. Wang, X. Zhang, H. Su, and J. Zhu. A Comprehensive Survey of Continual Learning: Theory, Method and Application. *PAMI*, 2024. URL `https://doi.ieeecomputersociety.org/10.1109/TPAMI.2024.3367329`.

F. Zenke, B. Poole, and S. Ganguli. Continual Learning Through Synaptic Intelligence. In *ICML*, 2017. URL `https://proceedings.mlr.press/v70/zenke17a.html`.

A The algorithm for Coreset VCL

By inspection of algorithm 1, there are several choices which are available to the modeller. In this section, we are interested in the coreset selection algortihm. Specifically, the algorithm permits us to add examples to and *remove examples from* the coreset. Nguyen et al. do not investigate whether an algorithm which both adds elements to, and removes elements from the coreset may be beneficial over one which only adds to the coreset. I conject that proper exploitation of this will better utilise memory, leading to better performance.

Any user who can run coreset VCL on n tasks with coresets of size c must have space to store $n \cdot c$ examples. Inspired by this, I investigate how best to use this memory. I propose a coreset selection algorithm where the coreset always contains data uniformly from all tasks which are seen, and frees up space by randomly dropping examples. Code for this is provided in algorithm 2.

Algorithm 1 Coreset VCL, figure taken from Nguyen et al.

Require: Prior $p(\theta)$.
Ensure: Variational and predictive distributions at each step $\{q_t(\theta), p(y^*|\mathbf{x}^*, \mathcal{D}_{1:t})\}_{t=1}^T$.

Initialize the coreset and variational approximation: $C_0 \leftarrow \emptyset$, $\tilde{q}_0 \leftarrow p$.
for $t = 1 \dots T$ **do**
  Observe the next dataset $\mathcal{D}_t$.
  $C_t \leftarrow$ update the coreset using C_{t-1} and $\mathcal{D}_t$.
  Update the variational distribution for non-coreset data points:

$$\tilde{q}_t(\theta) \leftarrow \arg\min_{q \in \mathcal{Q}} \mathrm{KL}\big(q(\theta) \parallel \tfrac{1}{Z}\tilde{q}_{t-1}(\theta)\, p(\mathcal{D}_t \cup C_{t-1} \setminus C_t|\theta)\big). \tag{1}$$

  Compute the final variational distribution (only used for prediction, and not propagation):

$$q_t(\theta) \leftarrow \arg\min_{q \in \mathcal{Q}} \mathrm{KL}\big(q(\theta) \parallel \tfrac{1}{Z}\tilde{q}_t(\theta) p(C_t|\theta)\big). \tag{2}$$

  Perform prediction at test input $\mathbf{x}^*$: $p(y^*|\mathbf{x}^*, \mathcal{D}_{1:t}) = \int q_t(\theta) p(y^*|\theta, \mathbf{x}^*) \mathrm{d}\theta$.
end for

Algorithm 2 Coreset selection algorithm

Require: Coreset size n, Coreset C_t, dataset $\mathcal{D}_{t+1}$
Ensure: Coreset contains data uniformly from all tasks
$C_{\mathcal{D}_{t+1}} \leftarrow$ random subset of $\mathcal{D}_{t+1}$ of size $\frac{n}{t+1}$
$\mathcal{D}_{t+1_C} \leftarrow$ random subset of C_t of size $\frac{n}{t+1}$
$C_{t+1} \leftarrow C_t \setminus \mathcal{D}_{t+1_C} \cup C_{\mathcal{D}_{t+1}}$

B Minimum required extension: VCL with a Gaussian likelihood

B.1 Deriving the loss for MSE error

Nguyen et al. do most of the derivation using an abstract likelihood p, meaning that the first half of the derivation remains unchanged. The objective of VCL is to find the variational distribution $\tilde{q}_{t+1}$ which minimises $\mathrm{KL}\left(q(\theta) \parallel \frac{1}{Z_t}\tilde{q}_t(\theta) \cdot p(\mathcal{D}_t \mid \theta)\right)$.

$$
\begin{aligned}
\tilde{q}_{t+1}(\theta) &= \arg\min_{q\in\mathcal{Q}} \mathrm{KL}\left(q(\theta) \parallel \frac{1}{Z_t}\tilde{q}_t(\theta)\cdot p(\mathcal{D}_t \mid \theta)\right) \\
&= \arg\min_{q\in\mathcal{Q}} \mathbb{E}_{\theta\sim q(\theta)}\left[\ln \frac{q(\theta)}{\frac{1}{Z_t}\tilde{q}_t(\theta)\cdot p(\mathcal{D}_t \mid \theta)}\right] \\
&= \arg\min_{q\in\mathcal{Q}} \mathbb{E}_{\theta\sim q(\theta)}\left[-\ln p(\mathcal{D}_t \mid \theta) + \ln \frac{q(\theta)}{\frac{1}{Z_t}\tilde{q}_t(\theta)\cdot}\right] \\
&= \arg\min_{q\in\mathcal{Q}} \mathbb{E}_{\theta\sim q(\theta)}\left[-\ln p(\mathcal{D}_t \mid \theta)\right] + \mathrm{KL}\left(q(\theta) \parallel \frac{1}{Z_t}\tilde{q}_t(\theta)\right) \\
&= \arg\min_{q\in\mathcal{Q}} \mathbb{E}_{\theta\sim q(\theta)}\left[-\ln \prod_{n=1}^{N_t} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(\frac{\left(f\left(\mathbf{x}_t^{(n)} \mid \theta\right) - y_t^{(n)}\right)^2}{2\sigma^2}\right)\right] + \mathrm{KL}\left(q(\theta) \parallel \frac{1}{Z_t}\tilde{q}_t(\theta)\right) \\
&= \arg\min_{q\in\mathcal{Q}} -\frac{1}{2}\ln(2\pi\sigma^2) + \sum_{n=1}^{N_t} \mathbb{E}_{\theta\sim q(\theta)}\left[\frac{\left(f\left(\mathbf{x}_t^{(n)} \mid \theta\right) - y_t^{(n)}\right)^2}{2\sigma^2}\right] + \mathrm{KL}\left(q(\theta) \parallel \frac{1}{Z_t}\tilde{q}_t(\theta)\right) \\
&= \arg\min_{q\in\mathcal{Q}} \sum_{n=1}^{N_t} \mathbb{E}_{\theta\sim q(\theta)}\left[\left(f\left(\mathbf{x}_t^{(n)} \mid \theta\right) - y_t^{(n)}\right)^2\right] + 2\sigma^2 \mathrm{KL}\left(q(\theta) \parallel \frac{1}{Z_t}\tilde{q}_t(\theta)\right)
\end{aligned}
$$

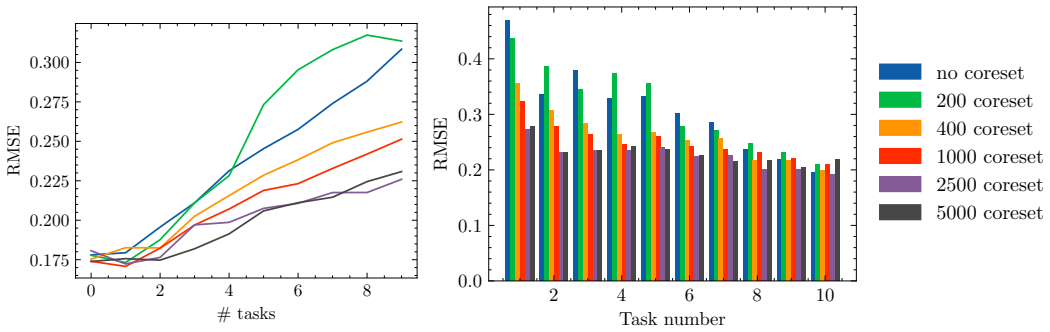
All terms in this are cheaply computable, thus this forms a suitable loss function. Concretely, the loss function which we want to *minimise* when applying VCL to tasks with Gaussian likelihoods is:

$$
\mathcal{L}_{\mathrm{VCL}}^t(q_t(\theta)) = \sum_{n=1}^{N_t} \mathbb{E}_{\theta\sim q_t(\theta)}\left[\left(f\left(\mathbf{x}_t^{(n)} \mid \theta\right) - y_t^{(n)}\right)^2\right] + 2\sigma^2 \cdot \mathrm{KL}(q_t(\theta) \parallel q_{t-1}(\theta))
$$

Note that choice of σ has potential to significantly impact behaviour of the function. It's not obvious how best to choose σ in the general case, and choice is highly likely to be dataset-dependent. Choosing bad values of σ could lead to bad convergence or catastrophic forgetting.

B.2 Results of VCL with MSE

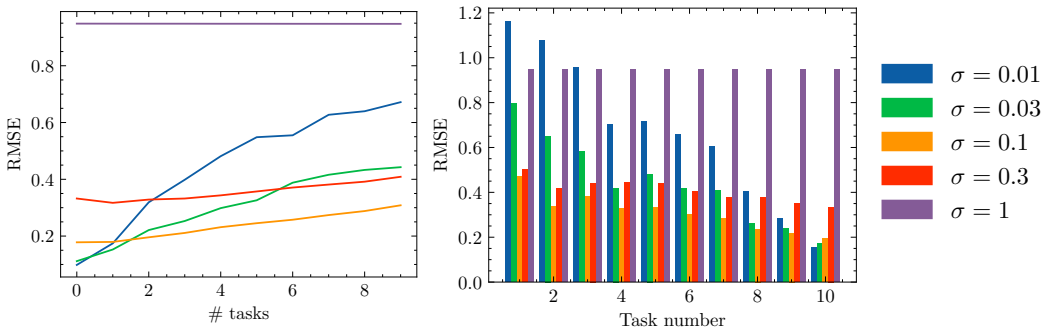
VCL still works with MSE, however, the additional data-dependent hyperparameter σ^2, the variance of the data, makes it more difficult to train. I present the results of my experiments with MSE error in figs. 3 and 4.



(a) RMSE of VCL averaged across tasks on permuted MNIST classification

(b) RMSE on each task of (coreset) VCL trained on 10 permuted MNIST tasks

Figure 3: RMSE of coreset VCL when trained on 10 tasks. Once again, we notice larger coresets lead to better performance.



(a) RMSE of VCL averaged across tasks on permuted MNIST classification

(b) RMSE on each task of (coreset) VCL trained on 10 permuted MNIST tasks

Figure 4: When training VCL using MSE loss, there is a hyperparameter σ for the standard deviation of the data. This figure shows how choice of σ affects how the VCL trains.

C Using a prior over accuracy to derive a prior over weights

This section provides a full derivation of how to convert a prior on the performance of the model into a suitable prior over weights. To the best of my knowledge, this is the first work to do this, and the first work to mathematically inform priors over weights.

The only prior which we actually have when training a model is on what performance it will achieve after training, say we have a prior that model accuracy is p. Guo et al. noticed that small models are often well-calibrated, whilst larger models are not. If our model is roughly calibrated, then confidence is an estimator for accuracy, so we can use our prior on accuracy as a prior on model confidence, *i.e.* the mean maximum probability we expect the model to output is p. If our model is not well-calibrated, then our prior must instead be over model confidence, which may be several percent higher than accuracy.

A normal prior over weights applied to data whose signals are normally distributed leads to normally distributed logits. If our logits are n-dimensional and are distributed according to $\mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$, we can easily compute what prior this is equivalent to placing on model confidence using the following code: `(`σ^2 `* torch.randn(k, n)).softmax(-1).amax(-1).mean()` for some large number k. We can use this code to quickly search for the variance σ^2 that corresponds to the prior want to place on model confidence.

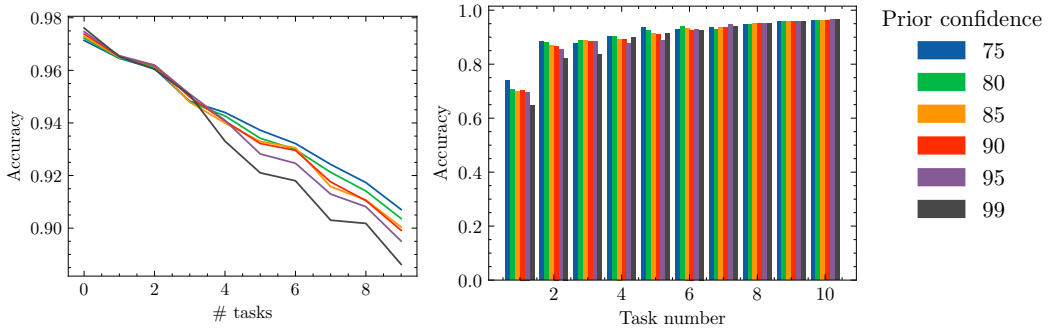
He et al. derived a distribution, known as the Kaiming-normal distribution, which preserves the variance of activations. This distribution is typically used for weight initialisation. For a network of depth d with normally distributed inputs, initialising weights to a Kaiming-normal distribution scaled by $\sigma^{\frac{2}{d}}$ will result in a prior that logits are distributed according to $\mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$. Thus, we have used a prior over model accuracy to derive a prior on model weights.

This does not impose a prior over biases, which we must manually select, and then rescale the prior over weights accordingly to ensure activations have the same variance. If our prior is that biases are distributed according to $\mathcal{N}(\mathbf{0}, \sigma_b^2\mathbf{I})$ for $\sigma_b < \sigma^{\frac{2}{d}}$, we would rescale the prior on the weights by $\sigma^{\frac{2}{d}} - \sigma_b^2$ instead of $\sigma^{\frac{2}{d}}$ to preserve the output distribution.

For VCL compounded Bayesian estimation errors mean the best prior is not the accuracy I found that using a prior corresponding to high confidence led to the highest accuracy on the first task (before training later tasks), see table 1; but after several rounds of recursive Bayesian estimation, priors corresponding to lower confidences were more performant, *i.e.* corresponding to 75% confidence. I conject that compounded estimation errors inherent in recursive Bayesian estimation led to this discrepancy. I provide a visualisation of the performance of different priors in fig. 5.

Confidence prior					
75%	80%	85%	90%	95%	99%
0.9714	0.9723	0.9729	0.9738	0.9747	0.9761

Table 1: Accuracy on the first task after training with VCL on only that task. Notice that the accuracy monotinocally increases with prior on confidence.



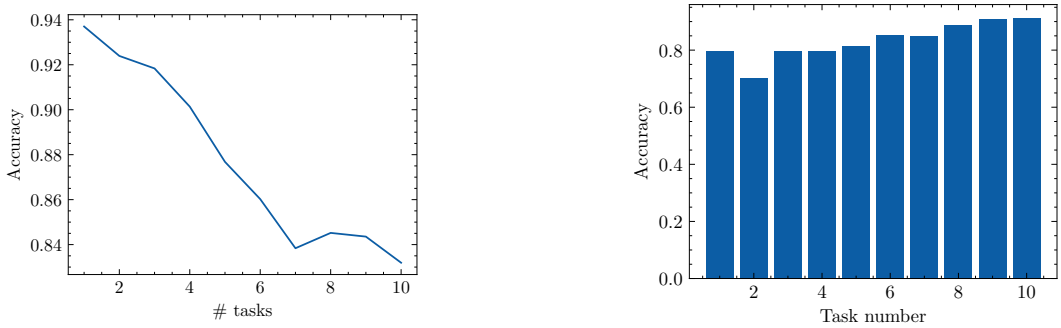
(a) Accuracy of VCL averaged across tasks on permuted MNIST classification

(b) Accuracy on each task of coreset VCL trained on 10 permuted MNIST tasks

Figure 5: Comparison of performance of VCL when initialised with priors corresponding to different model confidences. Priors corresponding to lower confidence corresponded to higher overall performance when trained. I conject that this is due to compounded estimation error.

D EWC Results

I present the results of training on the Permute MNIST dataset using EWC in fig. 6. I find that EWC is not as performant as VCL, even when VCL does not use coresets.



(a) Accuracy of EWC averaged across tasks on permuted MNIST classification

(b) Accuracy on each task of EWC trained on 10 permuted MNIST tasks

Figure 6: Performance of EWC on the Permuted MNIST task using manually tuned hyperparameter $\lambda = 120$. EWC is not as performant as VCL, and suffers from catastrophic forgetting more.